

Missing Values

```
In [38]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.impute import SimpleImputer

# 1. डेटा लोड करना
data = pd.read_csv('melb_data.csv')

# टारगेट (y) और प्रेडिक्टर्स (X) चुनना (केवल न्यूमेरिकल कॉलम)
y = data.Price
melb_predictors = data.drop(['Price'], axis=1)
X = melb_predictors.select_dtypes(exclude=['object'])

# डेटा को ट्रेनिंग और वैलिडेशन सेट में बांटना (80% ट्रेनिंग, 20% वैलिडेशन)
X_train, X_valid, y_train, y_valid = train_test_split(X, y, train_size=0.8, test_si

# 2. मॉडल की परफॉर्मेंस जाँचने के लिए फंक्शन (Score Function)
def score_dataset(X_train, X_valid, y_train, y_valid):
    model = RandomForestRegressor(n_estimators=10, random_state=0)
    model.fit(X_train, y_train)
    preds = model.predict(X_valid)
    return mean_absolute_error(y_valid, preds)
print(data.shape)
```

(13580, 21)

```
In [37]: cols_with_missing = [col for col in X_train.columns if X_train[col].isnull().any()]
cols_with_missing
```

```
Out[37]: ['Car', 'BuildingArea', 'YearBuilt']
```

मिसिंग डेटा से निपटने के 3 तरीके (Three Approaches)

Approach 1: जिन कॉलम में डेटा गायब है, उन्हें पूरी तरह हटा देना (Drop Columns)

क्या होता है: जिस कॉलम में एक भी वैल्यू मिसिंग हो, उस पूरे कॉलम को डिलीट कर देना।

नुकसान: इससे मॉडल बहुत सारी काम की जानकारी खो देता है, भले ही उस कॉलम की बाकी वैल्यूज़ सही हों।

```
In [34]: # 1. उन सभी कॉलम के नाम प्रिंट करना जिनमें मिसिंग (Null) वैल्यूज़ हैं
cols_with_missing = [
    col for col in X_train.columns if X_train[col].isnull().any()
]

print(f'-> कुल कितने कॉलम में मिसिंग वैल्यू हैं: {len(cols_with_missing)}')
print(f'-> उन कॉलम के नाम: {cols_with_missing}\n' + '=' * 60)
```

```
# 2. मिसिंग वैल्यू वाले कॉलम हटाने के बाद डेटा तैयार करना
reduced_X_train = X_train.drop(cols_with_missing, axis=1)
reduced_X_valid = X_valid.drop(cols_with_missing, axis=1)

# 3. नए रिड्यूसड डेटाफ्रेम का आकार (Shape) और शुरुआती डेटा प्रिंट करना
print('-> रिड्यूस होने के बाद X_train का आकार (Rows, Columns):', reduced_X_train.shape)
print('-> रिड्यूस होने के बाद X_valid का आकार (Rows, Columns):', reduced_X_valid.shape)
print('\n-> reduced_X_train के पहले 2 कॉलम/रो (हेड):')
print(reduced_X_train.head(2))

print("\n\n")
print("MAE from Approach 1 (Drop columns):")
print(score_dataset(reduced_X_train, reduced_X_valid, y_train, y_valid))
```

-> कुल कितने कॉलम में मिसिंग वैल्यू हैं: 3

-> उन कॉलम के नाम: ['Car', 'BuildingArea', 'YearBuilt']

=====

-> रिड्यूस होने के बाद X_train का आकार (Rows, Columns): (10864, 9)

-> रिड्यूस होने के बाद X_valid का आकार (Rows, Columns): (2716, 9)

-> reduced_X_train के पहले 2 कॉलम/रो (हेड):

	Rooms	Distance	Postcode	Bedroom2	Bathroom	Landsize	Latitude \
12167	1	5.0	3182	1	1	0	-37.85984
6524	2	8.0	3016	2	2	193	-37.85800

	Longitude	Propertycount
12167	144.9867	13240
6524	144.9005	6380

MAE from Approach 1 (Drop columns):
183550.22137772635

Approach 2: Imputation (गायब वैल्यू की जगह औसत/Mean भरना)

क्या होता है: मिसिंग वैल्यू की जगह उस कॉलम का औसत (Mean) या मध्य मान भर दिया जाता है।

फायदा: इससे पूरा कॉलम डिलीट नहीं करना पड़ता और मॉडल ज़्यादा सटीक (Accurate) बनता है।

```
In [39]: # =====
# APPROACH 2: Imputation (गायब वैल्यू की जगह Mean भरना)
# =====
my_imputer = SimpleImputer()
imputed_X_train = pd.DataFrame(my_imputer.fit_transform(X_train))
imputed_X_valid = pd.DataFrame(my_imputer.transform(X_valid))

# इम्प्यूटेशन के बाद कॉलम के नाम उड़ जाते हैं, उन्हें वापस जोड़ना
imputed_X_train.columns = X_train.columns
imputed_X_valid.columns = X_valid.columns
```

```
print("\nMAE from Approach 2 (Imputation):")
print(score_dataset(imputed_X_train, imputed_X_valid, y_train, y_valid))
```

MAE from Approach 2 (Imputation):

178166.46269899711

```
In [40]: # =====
# APPROACH 3: Imputation + गायब डेटा का ट्रैक रखना
# =====
# ओरिजिनल डेटा को सुरक्षित रखने के लिए कॉपी बनाना
X_train_plus = X_train.copy()
X_valid_plus = X_valid.copy()

# नए कॉलम जोड़ना जो बताएं कि कौन सा डेटा मिसिंग था
for col in cols_with_missing:
    X_train_plus[col + '_was_missing'] = X_train_plus[col].isnull()
    X_valid_plus[col + '_was_missing'] = X_valid_plus[col].isnull()

# अब इम्प्यूटेशन लागू करना
my_imputer = SimpleImputer()
imputed_X_train_plus = pd.DataFrame(my_imputer.fit_transform(X_train_plus))
imputed_X_valid_plus = pd.DataFrame(my_imputer.transform(X_valid_plus))

# कॉलम के नाम वापस सेट करना
imputed_X_train_plus.columns = X_train_plus.columns
imputed_X_valid_plus.columns = X_valid_plus.columns

print("\nMAE from Approach 3 (Extension to Imputation):")
print(score_dataset(imputed_X_train_plus, imputed_X_valid_plus, y_train, y_valid))
```

MAE from Approach 3 (Extension to Imputation):

178927.503183954

Shape of training data (num_rows, num_columns)

```
In [24]: print(X_train.shape)
```

(10864, 12)

Number of missing values in each column of training data

```
In [26]: missing_val_count_by_column = (X_train.isnull().sum())
missing_val_count_by_column
```

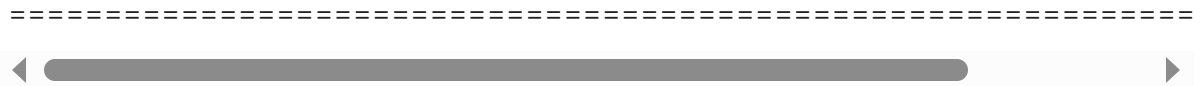
```
Out[26]: Rooms          0
Distance          0
Postcode          0
Bedroom2          0
Bathroom          0
Car               49
Landsize          0
BuildingArea      5156
YearBuilt         4307
Lattitude         0
Longitude         0
Propertycount     0
dtype: int64
```

```
In [27]: print(missing_val_count_by_column[missing_val_count_by_column > 0])
```

```
Car          49
BuildingArea 5156
YearBuilt    4307
dtype: int64
```

Conclusion

As is common, **imputing missing values (in Approach 2 and Approach 3) yielded better results**, relative to when we simply dropped columns with missing values (in Approach 1).



Categorical Variables

Score Function - Random Forest used in all 3 approach

```
In [41]: import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder
from sklearn.impute import SimpleImputer

# =====
# 1. डेटा लोड करना और ट्रेन-वैलिडेशन में बांटना
# =====
data = pd.read_csv('melb_data.csv')

# टारगेट (y) और फीचर्स (X) अलग करना
y = data['Price']
X = data.drop(['Price'], axis=1)

# 80% ट्रेनिंग और 20% वैलिडेशन डेटा में बांटना
X_train_full, X_valid_full, y_train, y_valid = train_test_split(
    X, y, train_size=0.8, test_size=0.2, random_state=0
```

```

)

# कम कार्डिनलिटी वाले कैटेगोरिकल कॉलम चुनना (Pandas 3.0 Compatible)
low_cardinality_cols = [
    cname for cname in X_train_full.select_dtypes(include=['object', 'str']).columns
    if X_train_full[cname].nunique() < 10
]

# न्यूमेरिकल कॉलम चुनना
numerical_cols = [
    cname for cname in X_train_full.select_dtypes(include=['int64', 'float64']).columns
]

# सिर्फ चुने हुए कॉलम (कैटेगोरिकल + न्यूमेरिकल) को रखना
my_cols = low_cardinality_cols + numerical_cols
X_train = X_train_full[my_cols].copy()
X_valid = X_valid_full[my_cols].copy()

print("-> low_cardinality_cols कैटेगोरिकल कॉलम:", low_cardinality_cols)
print("-> कुल न्यूमेरिकल कॉलम की संख्या:", len(numerical_cols))
print("=" * 60)

# =====
# मॉडल की परफॉर्मेंस जाँचने के लिए स्कोर फंक्शन (Score Function)
# =====
def score_dataset(X_t, X_v, y_t, y_v):
    model = RandomForestRegressor(n_estimators=100, random_state=0)
    model.fit(X_t, y_t)
    preds = model.predict(X_v)
    return mean_absolute_error(y_v, preds)

```

```

-> low_cardinality_cols कैटेगोरिकल कॉलम: ['Type', 'Method', 'Regionname']
-> कुल न्यूमेरिकल कॉलम की संख्या: 12
=====

```

Approach 1: Categorical Columns को पूरी तरह हटा देना (Drop Categorical Variables) क्या होता है: जिन कॉलम में टेक्स्ट/कैटेगरी होती है, उन्हें डिलीट कर देना।

नुकसान: अगर उस कॉलम में ज़रूरी जानकारी थी, तो मॉडल खराब परफॉर्म करेगा।

```

In [42]: # =====
# APPROACH 1: Drop Categorical Variables (कैटेगोरिकल कॉलम हटाना)
# =====
# सिर्फ न्यूमेरिकल कॉलम रखना और मिसिंग वैल्यू भरना
drop_X_train = X_train.select_dtypes(exclude=['object', 'str'])
drop_X_valid = X_valid.select_dtypes(exclude=['object', 'str'])

imputer = SimpleImputer(strategy='median')
drop_X_train_imputed = pd.DataFrame(imputer.fit_transform(drop_X_train), columns=drop_X_train.columns)
drop_X_valid_imputed = pd.DataFrame(imputer.transform(drop_X_valid), columns=drop_X_valid.columns)

mae_1 = score_dataset(drop_X_train_imputed, drop_X_valid_imputed, y_train, y_valid)
print(f"◆ Approach 1 (Drop Columns) MAE: ₹{mae_1:,.2f}")

```

◆ Approach 1 (Drop Columns) MAE: ₹169,749.04

Approach 2: Ordinal Encoding (क्रमबद्ध कोडिंग) क्या होता है: हर कैटेगरी की वैल्यू को एक नंबर दे देना (जैसे: Never = 0, Rarely = 1, Most days = 2, Every day = 3)।

कब इस्तेमाल करें: जब कैटेगरी के बीच कोई तय क्रम (Order) हो (जैसे साइज: Small, Medium, Large)।

```
In [20]: # =====
# APPROACH 2: Ordinal Encoding (ऑर्डिनल एन्कोडिंग)
# =====
ordinal_X_train = X_train.copy()
ordinal_X_valid = X_valid.copy()

# पहले कैटेगोरिकल मिसिंग वैल्यू को 'Unknown' से भरना ताकि एन्कोडर एरर न दे
ordinal_X_train[low_cardinality_cols] = ordinal_X_train[low_cardinality_cols].fillna('Unknown')
ordinal_X_valid[low_cardinality_cols] = ordinal_X_valid[low_cardinality_cols].fillna('Unknown')

# Ordinal Encoder लगाना
ordinal_encoder = OrdinalEncoder()
ordinal_X_train[low_cardinality_cols] = ordinal_encoder.fit_transform(ordinal_X_train[low_cardinality_cols])
ordinal_X_valid[low_cardinality_cols] = ordinal_encoder.transform(ordinal_X_valid[low_cardinality_cols])

# न्यूमेरिकल मिसिंग वैल्यू भरना
ordinal_X_train[numerical_cols] = imputer.fit_transform(ordinal_X_train[numerical_cols])
ordinal_X_valid[numerical_cols] = imputer.transform(ordinal_X_valid[numerical_cols])

mae_2 = score_dataset(ordinal_X_train, ordinal_X_valid, y_train, y_valid)
print(f"◆ Approach 2 (Ordinal Encoding) MAE: ₹{mae_2:,.2f}")
```

◆ Approach 2 (Ordinal Encoding) MAE: ₹162,793.95

Approach 3: One-Hot Encoding (वन-हॉट कोडिंग) क्या होता है: हर एक कैटेगरी के लिए नया कॉलम बना देना और उसमें 1 या 0 (हाँ या ना) भरना।

उदाहरण: अगर 'Color' कॉलम में 'Red', 'Yellow' है, तो यह 'Color_Red' और 'Color_Yellow' नाम से दो नए कॉलम बना देगा।

कब इस्तेमाल करें: जब कैटेगरी के बीच कोई तय क्रम न हो (जैसे कार का रंग या शहर के नाम)।

```
In [22]: # =====
# APPROACH 3: One-Hot Encoding (वन-हॉट एन्कोडिंग)
# =====
# कैटेगोरिकल मिसिंग वैल्यू भरना
OH_X_train = X_train.copy()
OH_X_valid = X_valid.copy()

OH_X_train[low_cardinality_cols] = OH_X_train[low_cardinality_cols].fillna('Missing')
OH_X_valid[low_cardinality_cols] = OH_X_valid[low_cardinality_cols].fillna('Missing')

# One-Hot Encoder लगाना
OH_encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
OH_cols_train = pd.DataFrame(OH_encoder.fit_transform(OH_X_train[low_cardinality_cols]))
OH_cols_valid = pd.DataFrame(OH_encoder.transform(OH_X_valid[low_cardinality_cols]))

# वन-हॉट एन्कोडेड कॉलम के सही नाम वापस देना
OH_cols_train.columns = OH_encoder.get_feature_names_out(low_cardinality_cols)
```

```
OH_cols_valid.columns = OH_encoder.get_feature_names_out(low_cardinality_cols)

# इंडेक्स रीसेट करना ताकि डेटा आपस में सही से जुड़ सके
OH_cols_train.index = OH_X_train.index
OH_cols_valid.index = OH_X_valid.index

# कैटेगोरिकल कॉलम हटाकर उनकी जगह वन-हॉट कॉलम जोड़ना
num_X_train = OH_X_train.drop(low_cardinality_cols, axis=1)
num_X_valid = OH_X_valid.drop(low_cardinality_cols, axis=1)

num_X_train[numerical_cols] = imputer.fit_transform(num_X_train[numerical_cols])
num_X_valid[numerical_cols] = imputer.transform(num_X_valid[numerical_cols])

OH_X_train_final = pd.concat([num_X_train, OH_cols_train], axis=1)
OH_X_valid_final = pd.concat([num_X_valid, OH_cols_valid], axis=1)

mae_3 = score_dataset(OH_X_train_final, OH_X_valid_final, y_train, y_valid)
print(f"💎 Approach 3 (One-Hot Encoding) MAE: ₹{mae_3:,.2f}")
print("=" * 60)
print("🎉 तीनों अप्रोच सफलतापूर्वक चल चुकी हैं!")
```

💎 Approach 3 (One-Hot Encoding) MAE: ₹161,879.65

=====

🎉 तीनों अप्रोच सफलतापूर्वक चल चुकी हैं!

Pipelines

Step 1: Define Preprocessing Steps (प्रीप्रोसेसिंग स्टेप्स तय करना)

इसमें हम ColumnTransformer का उपयोग करके न्यूमेरिकल और कैटेगोरिकल डेटा के लिए अलग-अलग ट्रांसफॉर्मर सेट करते हैं:

```
In [43]: from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder

# न्यूमेरिकल डेटा के लिए प्रीप्रोसेसिंग (मिसिंग वैल्यू भरना)
numerical_transformer = SimpleImputer(strategy='constant')

# कैटेगोरिकल डेटा के लिए प्रीप्रोसेसिंग (मिसिंग भरना + वन-हॉट एनकोडिंग)
categorical_transformer = Pipeline(
    steps=[
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False)),
    ]
)

# ColumnTransformer से दोनों को एक साथ जोड़ना
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, low_cardinality_cols),
    ]
)
```

```
]
)
```

क्या भरा जा रहा है?: यहाँ **strategy='constant'** का मतलब है कि न्यूमेरिकल कॉलम में जहाँ भी वैल्यू गायब (**NaN/Null**) होगी, वहाँ एक निश्चित (**Constant**) संख्या भरी जाएगी।

डिफ़ॉल्ट वैल्यू: यदि आप **strategy='constant'** के साथ कोई संख्या (जैसे **fill_value=0**) नहीं बताते हैं, तो पांडास/स्किट-लर्न डिफ़ॉल्ट रूप से 0 (शून्य) भर देता है। (नोट: कई बार Kaggle या प्रोजेक्ट्स में लोग **strategy='median'** का भी उपयोग करते हैं ताकि औसत या माधिका भरी जा सके)।

handle_unknown='ignore' के साथ: यह नया शहर आने पर क्रैश होने के बजाय शांत रहता है (Ignore करता है) और उस नए शहर के लिए बनने वाले सभी वन-हॉट कॉलम में चुपचाप सारे मान 0 (शून्य) भर देता है।

लेकिन, जब मॉडल ट्रेन होने के बाद भविष्य में नए डेटा पर Predict करता है (या वैलिडेशन डेटा पर चलता है), और अचानक कोई ऐसी नई कैटेगरी आ जाती है जो ट्रेनिंग के समय कभी नहीं देखी गई थी (जैसे नया शहर: 'Kolkata'), तो:

यह क्यों जरूरी है? यह मशीन लर्निंग मॉडल को Production में सुरक्षित रखता है ताकि भविष्य में कोई अनजान या नया डेटा आने पर आपका कोड अचानक फेल न हो।

Step 2: Define the Model (मॉडल तय करना)

इसके बाद हम अपने मशीन लर्निंग मॉडल (जैसे RandomForestRegressor) को डिफाइन करते हैं:

```
In [45]: from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(n_estimators=100, random_state=0)
```

Step 3: Create and Evaluate the Pipeline (पाइपलाइन बनाना और टेस्ट करना)

अब हम Pipeline क्लास की मदद से प्रीप्रोसेसिंग और मॉडल दोनों को आपस में जोड़ देते हैं:

```
In [47]: from sklearn.metrics import mean_absolute_error

# पाइपलाइन बनाना जिसमें पहले प्रीप्रोसेसिंग होगी और फिर मॉडल फिट होगा
my_pipeline = Pipeline(
    steps=[('preprocessor', preprocessor), ('model', model)]
)

# 1. ट्रेनिंग डेटा को एक ही लाइन में प्रीप्रोसेस और फिट करना
my_pipeline.fit(X_train, y_train)

# 2. वैलिडेशन डेटा पर सीधे प्रेडिक्शन निकालना (बिना अलग से मैनुअल प्रीप्रोसेसिंग किए!)
preds = my_pipeline.predict(X_valid)

# 3. स्कोर (MAE) चेक करना
```

```
score = mean_absolute_error(y_valid, preds)
print('Pipeline MAE:', score)
```

Pipeline MAE: 160679.18917034855

पाइपलाइन का सबसे बड़ा फायदा क्या है?

बिना पाइपलाइन के हमें ट्रेनिंग और वैलिडेशन डेटा पर बार-बार अलग से SimpleImputer और OneHotEncoder चलाना पड़ता था, जिसमें गलती होने की संभावना बहुत ज्यादा रहती थी। लेकिन Pipeline के साथ, हम सीधे रॉ (Unprocessed) वैलिडेशन डेटा को predict() में पास कर देते हैं, और पाइपलाइन अंदर ही अंदर खुद उसे प्रीप्रोसेस करके हमें सटीक रिजल्ट दे देती है।

Example case - How to use (Practical)

```
In [48]: import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

# 1. डेटा लोड करना
data = pd.read_csv('melb_data.csv')
y = data['Price']
X = data.drop(['Price'], axis=1)

# 2. ट्रेन और वैलिडेशन डेटा में बांटना (यहीं पर असल डेटा पास होता है)
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, train_size=0.8, test_size=0.2, random_state=0
)

# कॉलम चुनना
numerical_cols = [cname for cname in X_train.columns if X_train[cname].dtype in ['int64', 'float64']]
low_cardinality_cols = [cname for cname in X_train.columns if X_train[cname].nunique() < 10 and X_train[cname].dtype == 'object']

# 3. प्रीप्रोसेसिंग के नियम बनाना (इसमें डेटा नहीं दिया जाता, सिर्फ नियम तय होते हैं)
numerical_transformer = SimpleImputer(strategy='constant')

categorical_transformer = Pipeline(
    steps=[
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False)),
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, low_cardinality_cols),
    ]
)
```

```

# 4. पाइपलाइन बनाना (Preprocessor + Model)
model = RandomForestRegressor(n_estimators=50, random_state=0)

my_pipeline = Pipeline(
    steps=[('preprocessor', preprocessor), ('model', model)]
)

# =====
# 5. यहाँ असल में Train_X और Valid_X पास (दिए) जाते हैं और प्रिंट किए जाते हैं!
# =====

# ट्रेनिंग डेटा पर पाइपलाइन फिट करना
my_pipeline.fit(X_train, y_train)

# वैलिडेशन डेटा से प्रेडिक्शन निकालना (बिना अलग से मैनुअल प्रीप्रोसेसिंग किए)
preds = my_pipeline.predict(X_valid)

# रिजल्ट प्रिंट करना
print("-> प्रेडिक्शन के पहले 5 मान (Predictions):")
print(preds[:5])

print("\n-> मॉडल का Mean Absolute Error (MAE):")
print(mean_absolute_error(y_valid, preds))

```

```

-> प्रेडिक्शन के पहले 5 मान (Predictions):
[1762780.  897900.  602190. 1269610.  847655.]

```

```

-> मॉडल का Mean Absolute Error (MAE):
171338.0604958272

```

Cross Validation

अभी तक हम अपने डेटा को सिर्फ एक बार `train_test_split` की मदद से 80% ट्रेनिंग और 20% वैलिडेशन में बांटते आए हैं। लेकिन इसमें एक छोटी सी समस्या होती है: अगर हमारा वैलिडेशन सेट छोटा है, तो हमारे मॉडल का MAE स्कोर इस बात पर निर्भर कर सकता है कि उस 20% हिस्से में कौन-से घर (डेटा पॉइंट्स) आए हैं। इससे मॉडल के स्कोर में थोड़ा उतार-चढ़ाव (Variance) आ सकता है।

Cross-Validation इस समस्या का सबसे पक्का और शानदार समाधान है!

Cross-Validation क्या है? (What is it?) क्रॉस-वैलिडेशन में हम अपने डेटा को सिर्फ एक बार नहीं, बल्कि कई हिस्सों (Folds) में बांटते हैं (आमतौर पर 5 हिस्से, जिन्हें 5-fold CV कहा जाता है)।**

इसका काम करने का तरीका इस प्रकार है:

डेटा को Validation के लिए 5 बराबर भागों में बांटा जाता है।

पहला फोल्ड वैलिडेशन के लिए रखा जाता है और बाकी के 4 फोल्ड पर मॉडल को ट्रेन किया जाता है। इससे एक MAE स्कोर मिलता है।

इसके बाद दूसरा फोल्ड वैलिडेशन के लिए बनता है और बाकी पर ट्रेनिंग होती है। 4 यह प्रक्रिया 5 बार दोहराई जाती है, जिससे हमें 5 अलग-अलग MAE स्कोर मिल जाते हैं।

अंत में, हम इन सभी 5 स्कोर्स का औसत (Average/Mean) निकाल लेते हैं, जो हमें मॉडल की परफॉर्मेंस का एक बहुत ही सटीक और भरोसेमंद पैमाना देता है।

```
In [49]: from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score

# 1. पाइपलाइन बनाना (जो हमने पिछले चैप्टर में सीखी थी)
my_pipeline = Pipeline(steps=[('preprocessor', preprocessor),
                              ('model', RandomForestRegressor(n_estimators=50,
                                                              random_state=0))
                              ])

# 2. Cross-Validation चलाना (यहाँ cv=5 का मतलब है कि डेटा 5 हिस्सों में बंटेगा)
# scoring='neg_mean_absolute_error' का मतलब है कि स्किक्किट-लर्न MAE निकालता है
# (लेकिन वह इसे नेगेटिव वैल्यू में देता है, इसलिए हम आगे माइनस (-) लगाते हैं)
scores = -1 * cross_val_score(my_pipeline, X, y,
                              cv=5,
                              scoring='neg_mean_absolute_error')

print("Cross-validation scores:\n", scores)
print("Average MAE score (across experiments):")
print(scores.mean())
```

Cross-validation scores:

```
[214932.42281541 202764.67215022 193813.06024616 163794.29183638
166882.00946332]
```

Average MAE score (across experiments):

```
188437.2913023003
```

क्रॉस-वैलिडेशन का सबसे बड़ा फायदा क्या है?

अधिक भरोसा (Reliability): चूंकि इसमें मॉडल पूरे डेटा पर अलग-अलग एंगल से ट्रेन और टेस्ट हो चुका होता है, इसलिए इसका MAE स्कोर सिंगल train_test_split की तुलना में कहीं अधिक भरोसेमंद होता है।

बेहतर ट्यूनिंग: (Better Tuning) जब हम मॉडल के पैरामीटर्स (जैसे पेड़ों की संख्या या गहराई) बदलते हैं, तो क्रॉस-वैलिडेशन हमें सही आकलन देता है कि कौन सा मॉडल सच में बेहतर है।

XGBoost

अभी तक हमने Random Forest मॉडल का इस्तेमाल किया है, जो काफी अच्छा काम करता है। लेकिन मशीन लर्निंग प्रतियोगिताओं (जैसे Kaggle) में जब सबसे बेहतरीन स्कोर लाने की बात आती है, तो **XGBoost (Extreme Gradient Boosting)** राजा माना जाता है।

इसे बिल्कुल आसान हिंदी में समझें कि यह क्या है और इतना पावरफुल क्यों है:

XGBoost क्या है? (What is XGBoost?) XGBoost Gradient Boosting तकनीक का एक बहुत ही एडवांस और ऑप्टिमाइज़्ड रूप है।

सामान्य तरीका: साधारण मॉडल्स या तो अकेले काम करते हैं या अलग-अलग प्रेडिक्शन करते हैं।

Boosting का तरीका: इसमें मॉडल सीरियल (Sequential) तरीके से काम करते हैं। इसका मतलब है कि पहला मॉडल प्रेडिक्शन करता है, फिर जो गलतियां (Errors / Residuals) रह जाती हैं, दूसरा मॉडल उन गलतियों को सुधारने के लिए ट्रेन होता है, फिर तीसरा मॉडल बची हुई गलतियों को सुधारता है। इस तरह एक के बाद एक कई छोटे-छोटे मॉडल (Decision Trees) मिलकर एक बहुत ही ताकतवर मॉडल बनाते हैं।

Gradient boosting is a method that goes through cycles to iteratively add models into an ensemble.

It begins by initializing the ensemble with a single model, whose predictions can be pretty naive. (Even if its predictions are wildly inaccurate, subsequent additions to the ensemble will address those errors.)

```
In [ ]: !pip install xgboost
```

XGBRegressor + Cross-Validation चलाना (5-Fold CV)

```
In [1]: import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from xgboost import XGBRegressor

# 1. डेटा फ्रेश लोड करना
data = pd.read_csv('melb_data.csv')
y = data['Price']
X = data.drop(['Price'], axis=1)

# 2. ट्रेन और वैलिडेशन डेटा में बांटना
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, train_size=0.8, test_size=0.2, random_state=0
)

# 3. सही कॉलम ऑटोमैटिक चुनना (ताकि कोई कॉलम छूटे नहीं)
numerical_cols = [cname for cname in X_train.select_dtypes(include=['int64', 'float64'])]

low_cardinality_cols = [cname for cname in X_train.select_dtypes(include=['object'])
                        if X_train[cname].nunique() < 10]

# 4. प्रीप्रोसेसर (Preprocessor) दोबारा सही से बनाना
numerical_transformer = SimpleImputer(strategy='constant')

categorical_transformer = Pipeline(
    steps=[
        ('imputer', SimpleImputer(strategy='most_frequent')),
```

```

        ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False)),
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, low_cardinality_cols),
    ]
)

# 5. XGBoost मॉडल और पाइपलाइन तैयार करना
model = XGBRegressor(n_estimators=500, learning_rate=0.05, n_jobs=4, random_state=0)

my_pipeline = Pipeline(
    steps=[('preprocessor', preprocessor), ('model', model)]
)

# 6. अब यह बिना किसी एरर के आसानी से फिट हो जाएगा!
my_pipeline.fit(X_train, y_train)

# 7. प्रेडिक्शन और MAE निकालना
preds = my_pipeline.predict(X_valid)
print('✅ XGBoost MAE:', mean_absolute_error(y_valid, preds))

```

✅ XGBoost MAE: 153062.53125

Cross-Validation चलाना (5-Fold CV)

```

In [4]: # =====
# 5. Cross-Validation चलाना (5-Fold CV)
# =====
# scoring='neg_mean_absolute_error' का मतलब है कि एसकेकिट-लर्न MAE निकालेगा
# लेकिन उसे नेगेटिव (-) में देगा, इसलिए हम उसे -1 से गुणा करके पॉजिटिव बना लेते हैं।

from sklearn.model_selection import cross_val_score

scores = -1 * cross_val_score(
    my_pipeline,          # पाइपलाइन को cross_val_score के अंदर पास करने से डाटा
    X,
    y,
    cv=5,
    scoring='neg_mean_absolute_error'
)

print("📊 5 अलग-अलग फोल्ड के MAE स्कोर:", scores)
print("🌟 औसत (Average) MAE Score:", scores.mean())

```

📊 5 अलग-अलग फोल्ड के MAE स्कोर: [193290.28125 178825.265625 173985.125 147106.890625 152964.015625]

🌟 औसत (Average) MAE Score: 169234.315625

Data Leakage

मशीन लर्निंग में **Data Leakage (डेटा लीकेज)** एक ऐसी स्थिति है जब मॉडल को ट्रेनिंग के दौरान ऐसी जानकारी (Information) मिल जाती है जो उसे असली दुनिया में (या प्रेडिक्शन के वक्त) कभी नहीं मिलने वाली थी। इसकी वजह से मॉडल ट्रेनिंग के समय तो बहुत शानदार परफॉर्मेंस (अद्भुत स्कोर) दिखाता है, लेकिन जैसे ही उसे प्रोडक्शन या नए डेटा पर लगाया जाता है, वह बुरी तरह फेल हो जाता है।

मुख्य रूप से डेटा लीकेज 5 अलग-अलग स्थितियों (Conditions) में होता है:

1. Target Leakage (टारगेट लीकेज) यह तब होता है जब आपके फीचर्स (Features) में कोई ऐसा कॉलम या जानकारी शामिल होती है जो सीधे तौर पर Target Variable (जिसे प्रेडिक्ट करना है) के बारे में बता रही हो या भविष्य की हो (जो प्रेडिक्शन के समय उपलब्ध न हो)।

उदाहरण: मान लीजिए आप बैंक लोन डिफॉल्ट होने का प्रेडिक्शन कर रहे हैं। आपके डेटा में एक कॉलम है - "क्या कस्टमर ने लोन चुका दिया?"। अगर यह कॉलम आप इनपुट में दे देंगे, तो मॉडल को पहले से ही पता चल जाएगा कि कौन डिफॉल्ट हुआ और कौन नहीं, और मॉडल 100% सही रिजल्ट दिखाएगा—जो कि असली दुनिया में मुमकिन नहीं है क्योंकि नया कस्टमर आने पर यह जानकारी नहीं होगी।

2. Preprocessing Before Splitting (ट्रेन-टेस्ट स्प्लिट से पहले प्रीप्रोसेसिंग करना) यह डेटा लीकेज का सबसे आम तरीका है, जिसे हमने Pipelines के जरिए ठीक करना सीखा है।

स्थिति: यदि आप train_test_split करने से पहले पूरे डेटा पर SimpleImputer (मीन भरना), StandardScaler (स्केलिंग), या OneHotEncoder लगा देते हैं, तो वैलिडेट या टेस्ट सेट की जानकारी भी मीन/मीडियन निकालते वक्त ट्रेनिंग डेटा में शामिल हो जाती है।

नुकसान: मॉडल को पहले से ही टेस्ट डेटा के आंकड़ों की भनक लग जाती है।

बचाव: हमेशा डेटा को पहले स्प्लिट करें, और प्रीप्रोसेसिंग हमेशा सिर्फ X_train पर फिट होनी चाहिए। (या सबसे अच्छा तरीका: Pipeline का उपयोग करें)।

3. Improper Cross-Validation (गलत तरीके से क्रॉस-वैलिडेशन करना) **स्थिति:** क्रॉस-वैलिडेशन करते समय अगर आप पूरे डेटा पर फीचर सिलेक्शन (Feature Selection) या डेटा नॉर्मलाइजेशन (Normalization) फोल्ड में बांटने से पहले ही कर लेते हैं, तो लीकेज हो जाती है।

बचाव: फीचर सिलेक्शन और प्रीप्रोसेसिंग हमेशा क्रॉस-वैलिडेशन के हर एक फोल्ड के अंदर होनी चाहिए (इसीलिए हम Pipeline को cross_val_score के अंदर पास करते हैं ताकि लीकेज न हो)।

4. Temporal / Time-Series Leakage (समय-आधारित या टाइम-सीरीज डेटा में लीकेज) जब डेटा समय (Time/Date) से जुड़ा हो (जैसे शेयर बाजार के दाम या मौसम की भविष्यवाणी), तब लीकेज का खतरा बहुत ज्यादा होता है।

स्थिति: यदि आप समय के डेटा में train_test_split करते वक्त रैंडम स्प्लिट (random_state) कर देते हैं, तो भविष्य का डेटा ट्रेनिंग में चला जाता है और भूतकाल (Past) का डेटा टेस्ट में आ जाता है। इसे Future Leakage कहते हैं।

बचाव: टाइम-सीरीज डेटा में हमेशा क्रोनोलॉजिकल (कालक्रम के अनुसार - पुराने डेटा पर ट्रेन, नए डेटा पर टेस्ट) स्प्लिट करना चाहिए।

5. Duplicate Data Leakage (डुप्लीकेट डेटा स्प्लिट) स्थिति: यदि आपके पूरे डेटासेट में कुछ रो (Rows) डुप्लीकेट हैं और स्प्लिट करते समय एक जैसी रो X_train में भी चली गई और X_valid में भी आ गई।

नुकसान: मॉडल वैलिडेशन डेटा को देखकर कहेगा "अरे यह तो मैंने पहले ही देखा है!", जिससे स्कोर झूठा और बहुत अच्छा दिखाई देगा।

निष्कर्ष: डेटा लीकेज से बचने का सबसे अच्छा और आधुनिक तरीका है—Scikit-Learn Pipelines का उपयोग करना, क्योंकि पाइपलाइन खुद ध्यान रखती है कि प्रीप्रोसेसिंग सिर्फ ट्रेनिंग डेटा पर हो और वैलिडेशन डेटा पूरी तरह सुरक्षित रहे।

क्या आप इनमें से किसी खास प्रकार के लीकेज (जैसे टाइम-सीरीज या टारगेट लीकेज) के बारे में और गहराई से समझना चाहेंगे?

Hyperparameter Tuning क्या है?

जब हम मशीन लर्निंग मॉडल बनाते हैं, तो कुछ सेटिंग्स (Parameters) ऐसी होती हैं जिन्हें मॉडल खुद डेटा से नहीं सीखता, बल्कि हमें खुद तय करनी पड़ती हैं। इन्हें Hyperparameters कहते हैं। XGBoost में मॉडल की सटीकता (Accuracy) बढ़ाने के लिए मुख्य रूप से दो सेटिंग्स सबसे अहम होती हैं:

n_estimators (ट्रीज़ की संख्या): यह तय करता है कि मॉडल कितनी बार गलतियों को सुधारने की कोशिश करेगा।

learning_rate (सीखने की गति): यह तय करता है कि हर नया ट्री पिछले ट्री की गलती को कितना सुधारें।

अलग-अलग learning_rate को टेस्ट करने का कोड हम एक छोटे से लूप की मदद से देख सकते हैं कि अलग-अलग learning_rate पर हमारे मॉडल का MAE (Error) कितना आ रहा है:

अलग-अलग learning_rate पर क्या होता है ?

```
In [24]: # अलग-अलग Learning_rate की लिस्ट
my_list = [0.01, 0.05, 0.1, 0.2, 0.001]

for lr in my_list:
    # हर बार नया मॉडल और पाइपलाइन बनाना
    model = XGBRegressor(
        n_estimators=300, learning_rate=lr, n_jobs=4, random_state=0
    )

    my_pipeline = Pipeline(
        steps=[('preprocessor', preprocessor), ('model', model)]
    )

    # क्रॉस-वैलिडेशन से स्कोर निकालना
    scores = -1 * cross_val_score(
        my_pipeline, X, y, cv=3, scoring='neg_mean_absolute_error'
    )

    print(f'Learning Rate: {lr} → Average MAE: {scores.mean():.2f}')
```

Learning Rate: 0.01 → Average MAE: 199214.49
 Learning Rate: 0.05 → Average MAE: 178789.09
 Learning Rate: 0.1 → Average MAE: 179052.15
 Learning Rate: 0.2 → Average MAE: 183329.62
 Learning Rate: 0.001 → Average MAE: 381777.65

XGBRegressor + learning_rate + early_stopping_rounds

```
In [14]: import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from xgboost import XGBRegressor

# 1. डेटा लोड करना
data = pd.read_csv('melb_data.csv')
y = data['Price']
X = data.drop(['Price'], axis=1)

# 2. ट्रेन और वैलिडेशन में बांटना
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, train_size=0.8, test_size=0.2, random_state=0
)

# 3. कॉलम पहचानना
numerical_cols = [
    cname
    for cname in X_train.select_dtypes(include=['int64', 'float64']).columns
]
low_cardinality_cols = [
    cname
    for cname in X_train.select_dtypes(include=['object', 'str']).columns
    if X_train[cname].nunique() < 10
]

# 4. प्रीप्रोसेसर बनाना
numerical_transformer = SimpleImputer(strategy='constant')
categorical_transformer = Pipeline(
    steps=[
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False)),
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, low_cardinality_cols),
    ],
    remainder='drop',
```

```

)

# 5. सबसे पहले प्रीप्रोसेसर को train डेटा पर फिट करके train और valid दोनों को बदल लें
# (इससे वैलिडेशन डेटा भी पूरी तरह नंबरों में बदल जाएगा और कोई एरर नहीं आएगी)
X_train_transformed = preprocessor.fit_transform(X_train)
X_valid_transformed = preprocessor.transform(X_valid)

# 6. अब सीधे XGBRegressor मॉडल बनाएं (पाइपलाइन की जरूरत नहीं क्योंकि डेटा पहले ही साफ हो चुका है)
model = XGBRegressor(
    n_estimators=500,
    learning_rate=0.05,
    early_stopping_rounds=5,
    random_state=0,
)

# 7. मॉडल फिट करना (अब eval_set को साफ-सुथरा ट्रांसफॉर्म किया हुआ डेटा मिलेगा)
model.fit(
    X_train_transformed,
    y_train,
    eval_set=[(X_valid_transformed, y_valid)],
    verbose=False,
)

# 8. प्रेडिक्शन और MAE निकालना
preds = model.predict(X_valid_transformed)
print(
    '✅ Early Stopping के साथ XGBoost MAE:', mean_absolute_error(y_valid, preds)
)

```

✅ Early Stopping के साथ XGBoost MAE: 155617.21875

Cross-Validation चलाना (5-Fold CV)

```

In [20]: # =====
# 5. Cross-Validation चलाना (5-Fold CV)
# =====

from sklearn.model_selection import cross_val_score
from xgboost import XGBRegressor

# 1. मॉडल बनाते समय early_stopping_rounds हटा दें (क्योंकि CV में इसकी जरूरत नहीं होती)
model = XGBRegressor(
    n_estimators=300, # कोई फिक्स संख्या दे दें
    learning_rate=0.05,
    random_state=0,
)

# 2. पाइपलाइन तैयार करना
my_pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('model', model)])

# 3. Cross-Validation चलाना
scores = -1 * cross_val_score(
    my_pipeline, X, y, cv=5, scoring='neg_mean_absolute_error'
)

```

```
print('📊 5 अलग-अलग फोल्ड के MAE स्कोर:', scores)  
print('🌟 औसत (Average) MAE Score:', scores.mean())
```

📊 5 अलग-अलग फोल्ड के MAE स्कोर: [195035.046875 179416.640625 175014.875 148682.7
5 153507.609375]
🌟 औसत (Average) MAE Score: 170331.384375

In []: